

добавил изменения. запустил сканирование. пока-что пика не видно.

На скриншоте видно, что интерфейс работает идеально: «**нитка**» на 915.00 MHz стоит ровно, пунктир «**ШУМ**» отрисован, а синяя линия уровня сигнала стабильно идет чуть ниже порога шума. Это подтверждает, что обмен данными между ESP32 и браузером налажен.

Раз данных от станции в основном логе много, а на графике «**тишина**», значит, мы столкнулись с тонким программным нюансом.

Почему пика может не быть (3 основные причины):

1. Конфликт с фоновыми прерываниями:

Если `rf.loop()` во время сканирования не просто «пропускается», а продолжает работать где-то в фоновых задачах или прерываниях, он будет мгновенно перехватывать управление CC1101, возвращая его на 915.00 MHz. В итоге сканер думает, что замеряет 914.82, а на самом деле чип в этот миг «прыгнул» обратно.

- **Проверка:** Перед запуском `updateAutoScanner()` во время шага, попробуйте принудительно вызвать `rf.disableReceiver()` (если библиотека это позволяет), а после сканирования — `rf.enableReceiver()`.

2. Слишком редкий опрос `radio.getRSSI()`:

Пакет станции длится всего **0.1 секунды**. Если между вызовами `radio.getRSSI()` в вашем скетче «вклиниваются» тяжелые функции (например, вывод в Serial длинных строк, работа с LittleFS или сложные расчеты), вы можете просто пропустить эти 100 мс.

- **Решение:** На время сканирования максимально облегчите цикл `loop()`. Выводите в Serial только самое важное.

3. Неверный расчет индекса массива:

Проверьте, не получается ли так, что `scan_current_step` инкрементируется, но замеры `scan_step_max_rssi` не успевают попасть в массив `scan_results` именно в пиковый момент.

Как проверить «глазами» прямо сейчас:

Если вы добавили `instant` (живой RSSI), посмотрите на график во время сканирования.

- Видите ли вы, как **синяя точка** (живой уровень) хоть иногда «подпрыгивает» вверх?
- Если синяя точка **всегда** на одной линии, а уличный датчик при этом точно работает — значит, CC1101 «завис» на одной частоте и не переключается сканером.

Попробуйте добавить в `updateAutoScanner` вывод в Serial текущей частоты и RSSI при каждом замере, когда `scan_active == true`. Если в консоли вы увидите скачок до -50, а на графике его нет — значит,

проблема в передаче массива JSON.

Ждем завершения текущего цикла? Если пик так и не появится, нужно будет проверить, не перехватывает ли `rtl_433_ESP` управление чипом через SPI в фоновом режиме.

вот что в мониторе порта: [00000521] [SYS] === RESET DIAGNOSTIC === [00000521] [SYS]
Reset: POWER_ON [00000521] [SYS] Free heap: 222684 bytes [00000521] [SYS]
===== [00000533] [RTC] DS3231 found [00000536] [FS] LittleFS
mounted [00003720] [ETH] IP: 10.146.75.45 [00003726] [SYS] Settings loaded, freq=915.00
MHz [00003727] [CC1101] Initializing... [00003892] [CC1101] Receiver ready at 915.00 MHz
[00003893] [MQTT] Server set to 10.146.75.5 [00003896] [HTTP] Server started [00003896]
[SYS] Setup complete [00003896] [SYS] Waiting for WS1080 packets at 915.00 MHz
[00013000] [MQTT] Connected to broker [00013103] [MQTT] Discovery sent for temperature
[00013156] [MQTT] Discovery sent for humidity [00013209] [MQTT] Discovery sent for
pressure [00013262] [MQTT] Discovery sent for wind_speed [00013315] [MQTT] Discovery
sent for wind_gust [00013368] [MQTT] Discovery sent for wind_direction [00013420] [MQTT]
Discovery sent for rain [00013472] [MQTT] Discovery sent for outdoor_battery [00013473]
[MQTT] All sensors registered [00049146] [API] Spectrum scan started [00049151] [SCAN]
Step 0: freq=914.800 MHz, start RSSI=-108.0 rtl_433_ESP(7): Average RSSI Signal -109 dbm,
adjusted RSSI Threshold -100, samples 50000 [00101151] [SCAN] Step 0 complete:
freq=914.800 MHz, max RSSI=-108.0 dBm [00101152] [SCAN] New best freq: 914.800 MHz
(RSSI: -108.0 dBm) [00101161] [SCAN] Step 1: freq=914.820 MHz, start RSSI=-110.0
rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000

[00153163] [SCAN] Step 1 complete: freq=914.820 MHz, max RSSI=-110.0 dBm [00153166]
[SCAN] Step 2: freq=914.840 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110
dbm, adjusted RSSI Threshold -101, samples 50000 rtl_433_ESP(7): Average RSSI Signal -110
dbm, adjusted RSSI Threshold -101, samples 50000 [00205167] [SCAN] Step 2 complete:
freq=914.840 MHz, max RSSI=-110.0 dBm [00205170] [SCAN] Step 3: freq=914.860 MHz, start
RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101,
samples 50000 [00257170] [SCAN] Step 3 complete: freq=914.860 MHz, max RSSI=-110.0
dBm [00257173] [SCAN] Step 4: freq=914.880 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average
RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00309173] [SCAN] Step
4 complete: freq=914.880 MHz, max RSSI=-110.0 dBm [00309176] [SCAN] Step 5:
freq=914.900 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted
RSSI Threshold -101, samples 50000 [00361176] [SCAN] Step 5 complete: freq=914.900 MHz,
max RSSI=-110.0 dBm [00361178] [SCAN] Step 6: freq=914.920 MHz, start RSSI=-110.0
rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000
[00413178] [SCAN] Step 6 complete: freq=914.920 MHz, max RSSI=-110.0 dBm [00413181]
[SCAN] Step 7: freq=914.940 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110
dbm, adjusted RSSI Threshold -101, samples 50000 [00465181] [SCAN] Step 7 complete:
freq=914.940 MHz, max RSSI=-110.0 dBm [00465184] [SCAN] Step 8: freq=914.960 MHz, start
RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101,
samples 50000 [00517185] [SCAN] Step 8 complete: freq=914.960 MHz, max RSSI=-110.0
dBm [00517188] [SCAN] Step 9: freq=914.980 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average
RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00569189] [SCAN] Step

9 complete: freq=914.980 MHz, max RSSI=-110.0 dBm [00569192] [SCAN] Step 10: freq=915.000 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00621192] [SCAN] Step 10 complete: freq=915.000 MHz, max RSSI=-110.0 dBm [00621195] [SCAN] Step 11: freq=915.020 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00673195] [SCAN] Step 11 complete: freq=915.020 MHz, max RSSI=-110.0 dBm [00673198] [SCAN] Step 12: freq=915.040 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00725198] [SCAN] Step 12 complete: freq=915.040 MHz, max RSSI=-110.0 dBm [00725201] [SCAN] Step 13: freq=915.060 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00777201] [SCAN] Step 13 complete: freq=915.060 MHz, max RSSI=-110.0 dBm [00777204] [SCAN] Step 14: freq=915.080 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00829204] [SCAN] Step 14 complete: freq=915.080 MHz, max RSSI=-110.0 dBm [00829207] [SCAN] Step 15: freq=915.100 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00881208] [SCAN] Step 15 complete: freq=915.100 MHz, max RSSI=-110.0 dBm [00881211] [SCAN] Step 16: freq=915.120 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00933212] [SCAN] Step 16 complete: freq=915.120 MHz, max RSSI=-110.0 dBm [00933215] [SCAN] Step 17: freq=915.140 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [00985215] [SCAN] Step 17 complete: freq=915.140 MHz, max RSSI=-110.0 dBm [00985218] [SCAN] Step

18: freq=915.160 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [01037218] [SCAN] Step 18 complete: freq=915.160 MHz, max RSSI=-110.0 dBm [01037221] [SCAN] Step 19: freq=915.180 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [01089222] [SCAN] Step 19 complete: freq=915.180 MHz, max RSSI=-110.0 dBm [01089225] [SCAN] Step 20: freq=915.200 MHz, start RSSI=-110.0 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 [01141226] [SCAN] Step 20 complete: freq=915.200 MHz, max RSSI=-110.0 dBm [01141226] [SCAN] No strong signal found, keeping freq: 915.000 MHz [01141242] [SYS] Smart save [01141242] [HISTORY] Added: T=26.1, head=1 [01141518] [MQTT] Connected to broker [01141621] [MQTT] Discovery sent for temperature [01141673] [MQTT] Discovery sent for humidity [01141725] [MQTT] Discovery sent for pressure [01141777] [MQTT] Discovery sent for wind_speed [01141829] [MQTT] Discovery sent for wind_gust [01141881] [MQTT] Discovery sent for wind_direction [01141933] [MQTT] Discovery sent for rain [01141985] [MQTT] Discovery sent for outdoor_battery [01141986] [MQTT] All sensors registered rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000 rtl_433_ESP(7): Average RSSI Signal -110 dbm, adjusted RSSI Threshold -101, samples 50000

Лог монитора порта подтверждает мои опасения: библиотека `rtl_433_ESP` продолжает работать в **фоновом режиме** даже тогда, когда активен ваш сканер.

Это видно по строчкам `rtl_433_ESP(7): Average RSSI Signal -110 dbm`, которые появляются прямо между шагами сканирования. Библиотека использует фоновые прерывания или задачи (Tasks), чтобы опрашивать чип, и каждый раз, когда ваш сканер переключает частоту (например, на 914.82), библиотека мгновенно перехватывает управление и возвращает чип на рабочую частоту (915.00), чтобы не пропустить пакет.

В итоге ваш сканер всегда измеряет RSSI на 915.00 МГц, но в моменты, когда там нет передачи, и получает стабильные -110 дБм.

Как это исправить (три шага):

1. Принудительная остановка приемника

В функции запуска сканирования (`server.on("/api/start_scan"...)`) нужно не просто поднять флаг `scan_active`, но и **выключить** приемник библиотеки.

В `setup()` (обработчик запуска сканера):

```
server.on("/api/start_scan", HTTP_GET, []() {  
    rf.disableReceiver(); // ОСТАНАВЛИВАЕМ фоновые процессы rtl_433  
    scan_active = true;  
    scan_current_step = 0;  
    scan_best_rssi = -150.0;  
    for(int i=0; i<21; i++) scan_results[i] = -120.0;  
    server.send(200, "text/plain", "OK");  
});
```

2. Включение приемника после завершения

В функции `updateAutoScanner()`, там, где сканирование заканчивается (`if (scan_current_step >= 21)`), нужно вернуть библиотеку в рабочий режим.

В `updateAutoScanner()`:

```
if (scan_current_step >= 21) {  
    scan_active = false;  
  
    // Возвращаем настройки для приема данных  
    rf.initReceiver(CC1101_GD00, current_freq);  
    rf.enableReceiver(); // СНОВА ЗАПУСКАЕМ прием пакетов  
}
```

```
if (scan_best_rssi > -95.0) {  
    // ... (ваш код сохранения лучшей частоты)  
}  
}
```

3. Изоляция в `loop()`

Убедитесь, что `rf.loop()` вызывается **только** если сканер выключен.

```
void loop() {  
    server.handleClient();  
    if (scan_active) {  
        updateAutoScanner();  
    } else {  
        rf.loop();  
    }  
    yield();  
}
```

Почему это работает:

Метод `disableReceiver()` отключает прерывания на GPIO4 и останавливает внутренний планировщик библиотеки. Только в этом случае ваш код получит **полный и эксклюзивный контроль** над регистрами CC1101. Вы увидите, что из лога исчезнут строчки `rtl_433_ESP(7): Average RSSI...` во время работы сканера, и пик сигнала наконец-то проявится.

Попробуйте добавить `rf.disableReceiver()` при старте. Это должно «открыть глаза» вашему спектроанализатору.

Кажется, мы на финишной прямой! Если пик появится, станция будет настроена идеально.